

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2

По дисциплине «Модели решения задач в интеллектуальных системах»

Тема: «Многослойный персептрон для задачи «исключающее ИЛИ» с числовыми классами: сравнение функций потерь MSE и Binary Cross-Entropy»

Выполнил:

Студент 4 курса

Группы ИИ-26

Кушнеревич Е. А.

Проверил:

Никитин С. К.

Брест 2026

Цель работы: Реализовать альтернативную функцию потерь – бинарную кросс-энтропию (Binary Cross-Entropy, BCE) – и провести сравнительный анализ обучения сети с MSE (Лабораторная работа №1) и с BCE по скорости сходимости, качеству обучения и устойчивости к выбору начальных весов.

Ход работы:

1. Взять за основу код многослойного персептрона 2-2-1 из Лабораторной работы №1.
 2. Поскольку BCE определена для выходов в диапазоне $(0; 1)$, а целевые значения в вашем варианте – произвольные числа c_0 и c_1 из диапазона $[-10; 10]$, реализовать нормализацию (масштабирование) целевых значений перед обучением и обратное преобразование выхода сети после обучения:
 - привести c_0 и c_1 к бинарным меткам 0 и 1;
 - обучать сеть на нормализованных целевых значениях с функцией потерь BCE;
 - при выводе результата преобразовывать выход сети $\hat{y} \in (0; 1)$ обратно в шкалу $[c_0; c_1]$.
 3. Реализовать корректный градиент для связки BCE + сигмоида на выходном слое; для скрытого слоя градиент передаётся по стандартному правилу цепочки.
 4. Обучить сеть на полной таблице истинности своего варианта (4 примера) в двух конфигурациях:
 - **Конфигурация А (базовая, из ЛР №1):** MLP 2-2-1, функция потерь MSE, целевые значения – исходные c_0/c_1 , выходной слой – сигмоида. Результаты можно взять из отчёта по ЛР №1 или, при необходимости, повторить обучение с фиксацией всех метрик.
 - **Конфигурация Б (новая):** MLP 2-2-1, функция потерь BCE, целевые значения – нормализованные $(0/1)$, с последующим пересчётом выхода в шкалу $[c_0; c_1]$.
- Критерий остановки для обеих конфигураций одинаков – суммарная ошибка $E_s \leq E_e$ (используйте то же значение E_e , что и в ЛР №1, либо согласуйте новое значение отдельно для MSE и для BCE, поскольку это разные по своей природе величины – обоснуйте выбор в отчёте).
5. Для каждой конфигурации зафиксировать:
 - количество эпох до достижения критерия остановки (либо факт недостижения за максимально допустимое число эпох);
 - итоговую суммарную ошибку;
 - ассигасу на всех 4 примерах таблицы истинности;
 - среднюю абсолютную ошибку в исходной шкале $[c_0; c_1]$.
 6. Провести не менее 5 запусков обучения для каждой конфигурации с разной случайной инициализацией весов (менять seed) и сравнить устойчивость сходимости: в скольких из 5 запусков сеть достигла критерия остановки, и какой разброс по числу эпох наблюдается.
 7. Построить общий график зависимости суммарной ошибки от номера эпохи для конфигурации А (MSE) и конфигурации Б (BCE) на одних осях (для представительного запуска каждой конфигурации).
 8. Реализовать блок визуализации результатов, включающий:
 - **График сходимости** – суммарная ошибка E_s от номера эпохи, конфигурации А и Б на одних осях, разными цветами/стилями линий, с обозначением порога E_e .
 - **Диаграмму разброса числа эпох по 5 запускам** (bar chart или scatter) для конфигурации А и конфигурации Б – наглядно показывающую устойчивость сходимости из п. 6 (например, столбики числа эпох для каждого из 5 seed, сгруппированные по конфигурации; запуски, не достигшие критерия

остановки, отметить отдельно, например штриховкой или отдельным цветом).

- **Визуализацию разделяющей поверхности скрытого слоя** – для представительного обученного экземпляра каждой конфигурации построить на плоскости (А, В) карту выходного значения сети (heatmap или contour plot) с наложенными точками обучающей выборки (4 примера таблицы истинности), закрашенными по истинному классу. Диапазон осей А и В – $[-10; 10]$ (весь допустимый диапазон входных значений). Для конфигурации Б цветовую шкалу можно строить как по нормализованному $\hat{y} \in (0; 1)$, так и по пересчитанному значению в шкале $[c0; c1]$ – выбрать более наглядный вариант и обосновать выбор.
- **Диаграмму сравнения точности восстановления шкалы** – столбчатую диаграмму средней абсолютной ошибки для конфигурации А и конфигурации Б, чтобы наглядно сопоставить точность регрессии к исходной шкале $[c0; c1]$.

Каждый график должен иметь подписанные оси, заголовок и легенду (где применимо). Разделяющую поверхность и графики точности рекомендуется разместить рядом (например, subplot 2×2 или отдельными рисунками) для удобства сравнения в отчёте.

9. Реализовать режим функционирования сети: пользователь вводит пару чисел (А, В) из диапазона $[-10; 10]$ (не обязательно совпадающих с $c0$ и $c1$). Сеть должна выводить:

- выход в нормализованной шкале $\hat{y} \in (0; 1)$ (с точностью до 4 знаков);
- выход, пересчитанный в исходную шкалу $[c0; c1]$ по формуле из п. 2;
- к какому классу ($c0$ или $c1$) ближе полученное значение.

Продемонстрировать работу режима функционирования на всех 4 примерах исходной таблицы истинности и не менее чем на 2–3 дополнительных парах (А, В), не входящих в обучающую выборку.

Вариант 8 ($c0 = -8, c1 = 3$)

А	В	$A \oplus B$
-8	-8	-8
-8	3	3
3	-8	3
3	3	-8

Код программы:

```
import os
import sys
import numpy as np
import matplotlib
import matplotlib.pyplot as plt

c0, c1 = -8.0, 3.0
X = np.array([[-8, -8], [-8, 3], [3, -8], [3, 3]], dtype=float)
Y_raw = np.array([[-8], [3], [3], [-8]], dtype=float)

def normalize(x, lo, hi): return (x - lo) / (hi - lo)
def denormalize(y, lo, hi): return lo + y * (hi - lo)

Xn = normalize(X, c0, c1)
Yn = normalize(Y_raw, c0, c1)
```

```

EPS = 1e-12
def sigmoid(s): return 1.0 / (1.0 + np.exp(-s))
def dsigmoid(y): return y * (1.0 - y)

class MLP221:
    def __init__(self, loss="mse", lr=0.5, seed=7):
        rng = np.random.RandomState(seed)
        self.W1 = rng.uniform(-0.1, 0.1, (2, 2)); self.T1 = rng.uniform(-0.1, 0.1, (1, 2))
        self.W2 = rng.uniform(-0.1, 0.1, (2, 1)); self.T2 = rng.uniform(-0.1, 0.1, (1, 1))
        self.lr, self.loss = lr, loss
        self.rng = np.random.RandomState(seed + 1000)

    def forward(self, x):
        x = np.asarray(x, dtype=float).reshape(1, -1)
        y1 = sigmoid(x @ self.W1 - self.T1)
        y2 = sigmoid(y1 @ self.W2 - self.T2)
        return y1, y2

    def error(self, y, e):
        if self.loss == "mse":
            return 0.5 * np.sum((y - e) ** 2)
        y = np.clip(y, EPS, 1 - EPS)
        return -np.sum(e * np.log(y) + (1 - e) * np.log(1 - y))

    def train_step(self, x, e):
        x, e = x.reshape(1, -1), e.reshape(1, -1)
        y1, y2 = self.forward(x)
        if self.loss == "mse":
            delta2 = (y2 - e) * dsigmoid(y2)
        else:
            delta2 = (y2 - e)
        delta1 = (delta2 @ self.W2.T) * dsigmoid(y1)
        self.W2 -= self.lr * (y1.T @ delta2); self.T2 += self.lr * delta2
        self.W1 -= self.lr * (x.T @ delta1); self.T1 += self.lr * delta1
        return self.error(y2, e)

    def predict(self, x): return self.forward(x)[1][0, 0]

    def fit(self, X, Y, Ee, max_epochs=20000):
        hist = []
        for epoch in range(1, max_epochs + 1):
            idx = self.rng.permutation(len(X))
            Es = sum(self.train_step(X[i], Y[i]) for i in idx)
            hist.append(Es)
            if Es <= Ee:
                return epoch, Es, True, hist
        return max_epochs, Es, False, hist

    def accuracy(m, Xn, Y, thr=0.5):
        return np.mean([int(m.predict(x) >= thr) == int(y[0]) for x, y in zip(Xn, Y)])

    def mae_orig(m, Xn, Yraw):
        p = np.array([denormalize(m.predict(x), c0, c1) for x in Xn])
        return float(np.mean(np.abs(p - Yraw.ravel()))))

    def final_Es(m, Xn, Y):
        return sum(m.error(m.forward(x)[1], y.reshape(1, -1)) for x, y in zip(Xn, Y))

    def infer(m, A, B, name):
        yn = m.predict(normalize(np.array([A, B], dtype=float), c0, c1))

```

```

    yh = denormalize(yn, c0, c1)
    cls = c0 if abs(yh - c0) <= abs(yh - c1) else c1
    print(f"[{name}] A={A:g}, B={B:g} -> y_norm={yn:.4f}, y_hat={yh:.4f}, ближе к классу {cls:g}")

CONFIGS = {
    "A (MSE)": dict(loss="mse", Ee=0.01),
    "B (BCE)": dict(loss="bce", Ee=0.05),
}

SEEDS = [1, 2, 3, 4, 5]
MAX_EPOCHS = 20000

def run(cfg, seed):
    m = MLP221(loss=cfg["loss"], seed=seed)
    ep, Es, ok, hist = m.fit(Xn, Yn, cfg["Ee"], MAX_EPOCHS)
    return m, dict(seed=seed, epochs=ep, Es=final_Es(m, Xn, Yn), ok=ok,
                  acc=accuracy(m, Xn, Yn), mae=mae_orig(m, Xn, Y_raw), hist=hist)

if __name__ == "__main__":
    res, models = {}, {}
    for name, cfg in CONFIGS.items():
        res[name], models[name] = [], []
        for s in SEEDS:
            m, r = run(cfg, s); res[name].append(r); models[name].append(m)
            print(f"\n Конфигурация {name}, Ee={cfg['Ee']} ")
            print("seed | эпох | Es | acc | MAE[c0;c1] | достигнут Ee")
            for r in res[name]:
                print(f"{r['seed']}>4} | {r['epochs']>5} | {r['Es']:.5f} | {r['acc']:.2f} | {r['mae']:.4f} | {r['ok']}]")
            ep = [r['epochs'] for r in res[name]]
            print(f"Совлось: {sum(r['ok'] for r in res[name])}/5; эпохи min={min(ep)}, max={max(ep)}, "
                  f"mean={np.mean(ep):.0f}, std={np.std(ep):.0f}")

    rep = {}
    for name in CONFIGS:
        ok = [i for i, r in enumerate(res[name]) if r['ok']] or list(range(len(SEEDS)))
        i = sorted(ok, key=lambda k: res[name][k]['epochs'])[len(ok) // 2]
        rep[name] = i
        colors = {"A (MSE)": "tab:blue", "B (BCE)": "tab:red"}

    plt.figure(figsize=(8, 5))
    for name, cfg in CONFIGS.items():
        h = res[name][rep[name]]["hist"]
        plt.plot(range(1, len(h) + 1), h, color=colors[name], ls="-" if "MSE" in name else "--",
                label=f"{name}, seed={res[name][rep[name]]['seed']}")
        plt.axhline(cfg["Ee"], color=colors[name], ls=":", alpha=.7, label=f"Ee {name} = {cfg['Ee']}")
    plt.yscale("log"); plt.xscale("log")
    plt.xlabel("Эпоха"); plt.ylabel("Суммарная ошибка Es"); plt.title("Сходимость: MSE vs BCE")
    plt.legend(); plt.grid(alpha=.3); plt.tight_layout(); plt.savefig("1_convergence.png",
        dpi=150); plt.show(); plt.close()

    plt.figure(figsize=(8, 5)); w = 0.38; xs = np.arange(len(SEEDS))
    for k, name in enumerate(CONFIGS):
        for j, r in enumerate(res[name]):
            plt.bar(xs[j] + (k - .5) * w, r["epochs"], w, color=colors[name],

```

```

        hatch=None if r["ok"] else "/", edgecolor="k",
        label=name if j == 0 else None, alpha=1 if r["ok"] else .5)
plt.bar(0, 0, color="w", edgecolor="k", hatch="/", label="Не достигнут Ее")
plt.xticks(xs, [f"seed {s}" for s in SEEDS]); plt.yscale("log")
plt.xlabel("Запуск"); plt.ylabel("Число эпох"); plt.title("Устойчивость сходимости (5
запусков)")
plt.legend(); plt.grid(axis="y", alpha=.3); plt.tight_layout();
plt.savefig("2_epochs.png", dpi=150); plt.show(); plt.close()

g = np.linspace(-10, 10, 200); GA, GB = np.meshgrid(g, g)
fig, axs = plt.subplots(1, 2, figsize=(12, 5))
for ax, name in zip(axs, CONFIGS):
    m = models[name][rep[name]]
    Z = np.array([[denormalize(m.predict(normalize(np.array([a, b])), c0, c1)), c0, c1)
                  for a in g] for b in g])
    cs = ax.contourf(GA, GB, Z, levels=30, cmap="viridis", vmin=c0, vmax=c1)
    ax.contour(GA, GB, Z, levels=[(c0 + c1) / 2], colors="w", linestyles="--")
    for xi, yi in zip(X, Y_raw.ravel()):
        ax.scatter(*xi, c=[1, .85, 0] if yi == c1 else [1, .2, .2], s=140,
edgecolors="k")
    ax.scatter([], [], c="gold", edgecolors="k", label=f"класс {c1:g}")
    ax.scatter([], [], c="tab:red", edgecolors="k", label=f"класс {c0:g}")
    ax.set_xlabel("A"); ax.set_ylabel("B"); ax.set_title(f"Выход сети, шкала
[{c0:g};{c1:g}] – {name}")
    ax.legend(loc="lower right"); fig.colorbar(cs, ax=ax, label="ŷ в шкале [c0; c1]")
plt.tight_layout(); plt.savefig("3_surface.png", dpi=150); plt.show(); plt.close()

plt.figure(figsize=(6, 5))
means = [np.mean([r["mae"] for r in res[n]]) for n in CONFIGS]
stds = [np.std([r["mae"] for r in res[n]]) for n in CONFIGS]
bars = plt.bar(list(CONFIGS), means, yerr=stds, capsize=5, color=[colors[n] for n in
CONFIGS], label="MAE (среднее ± std, 5 seed)")
for b, v in zip(bars, means): plt.text(b.get_x() + b.get_width()/2, v, f"{v:.4f}",
ha="center", va="bottom")
plt.xlabel("Конфигурация"); plt.ylabel("Средняя абсолютная ошибка в [c0; c1]")
plt.title("Точность восстановления шкалы"); plt.legend(); plt.tight_layout()
plt.savefig("4_mae.png", dpi=150); plt.show(); plt.close()

tests = [tuple(x) for x in X] + [(0, 0), (-5, 7), (9.5, -3)]
for name in CONFIGS:
    print(f"\n Функционирование, конфигурация {name} ")
    for a, b in tests: infer(models[name][rep[name]], a, b, name)

if sys.stdin.isatty():
    m = models["B (BCE)"][rep["B (BCE)"]]
    while True:
        raw = input("Введите A B из [-10;10] (пусто – выход): ").strip()
        if not raw: break
        try:
            a, b = map(float, raw.split()); assert -10 <= a <= 10 and -10 <= b <= 10
        except Exception:
            print("Нужны два числа из [-10; 10]"); continue
        infer(m, a, b, "BCE")

```

Результат тестирования:

Конфигурация А (MSE), $E_{\epsilon}=0.01$

seed | эпох | E_s | асс | MAE[c0;c1] | достигнут E_{ϵ}

1	2749	0.00991	1.00	0.7641	True
2	20000	0.33377	0.75	3.6954	False
3	3054	0.00992	1.00	0.7635	True
4	2856	0.00990	1.00	0.7641	True
5	2862	0.00992	1.00	0.7645	True

Сошлось: 4/5; эпохи min=2749, max=20000, mean=6304, std=6849

Конфигурация В (BCE), $E_{\epsilon}=0.05$

seed | эпох | E_s | асс | MAE[c0;c1] | достигнут E_{ϵ}

1	4117	0.04937	1.00	0.1348	True
2	20000	1.91039	0.75	3.6868	False
3	20000	1.90999	0.75	3.6784	False
4	872	0.04943	1.00	0.1350	True
5	943	0.04937	1.00	0.1349	True

Сошлось: 3/5; эпохи min=872, max=20000, mean=9186, std=8907

Функционирование, конфигурация А (MSE)

[A (MSE)] A=-8, B=-8 -> $y_{\text{norm}}=0.0568$, $y_{\text{hat}}=-7.3750$, ближе к классу -8

[A (MSE)] A=-8, B=3 -> $y_{\text{norm}}=0.9333$, $y_{\text{hat}}=2.2658$, ближе к классу 3

[A (MSE)] A=3, B=-8 -> $y_{\text{norm}}=0.9333$, $y_{\text{hat}}=2.2658$, ближе к классу 3

[A (MSE)] A=3, B=3 -> $y_{\text{norm}}=0.0877$, $y_{\text{hat}}=-7.0352$, ближе к классу -8

[A (MSE)] A=0, B=0 -> $y_{\text{norm}}=0.5945$, $y_{\text{hat}}=-1.4605$, ближе к классу 3

[A (MSE)] A=-5, B=7 -> $y_{\text{norm}}=0.3190$, $y_{\text{hat}}=-4.4906$, ближе к классу -8

[A (MSE)] A=9.5, B=-3 -> $y_{\text{norm}}=0.0786$, $y_{\text{hat}}=-7.1351$, ближе к классу -8

Функционирование, конфигурация В (BCE)

[B (BCE)] A=-8, B=-8 -> $y_{\text{norm}}=0.0092$, $y_{\text{hat}}=-7.8985$, ближе к классу -8

[B (BCE)] A=-8, B=3 -> $y_{\text{norm}}=0.9890$, $y_{\text{hat}}=2.8791$, ближе к классу 3

[B (BCE)] A=3, B=-8 -> $y_{\text{norm}}=0.9890$, $y_{\text{hat}}=2.8791$, ближе к классу 3

[B (BCE)] A=3, B=3 -> $y_{\text{norm}}=0.0178$, $y_{\text{hat}}=-7.8038$, ближе к классу -8

[B (BCE)] A=0, B=0 -> $y_{\text{norm}}=0.6687$, $y_{\text{hat}}=-0.6445$, ближе к классу 3

[B (BCE)] A=-5, B=7 -> $y_{\text{norm}}=0.1825$, $y_{\text{hat}}=-5.9920$, ближе к классу -8

[B (BCE)] A=9.5, B=-3 -> $y_{\text{norm}}=0.0156$, $y_{\text{hat}}=-7.8288$, ближе к классу -8

Введите А В из [-10;10] (пусто — выход): -8 8

[BCE] A=-8, B=8 -> $y_{\text{norm}}=0.6684$, $y_{\text{hat}}=-0.6474$, ближе к классу 3

Введите А В из [-10;10] (пусто — выход): -8 -3

[BCE] A=-8, B=-3 -> $y_{\text{norm}}=0.7323$, $y_{\text{hat}}=0.0554$, ближе к классу 3

Введите А В из [-10;10] (пусто — выход): -3 8

[BCE] A=-3, B=8 -> $y_{\text{norm}}=0.0256$, $y_{\text{hat}}=-7.7181$, ближе к классу -8

Введите А В из [-10;10] (пусто — выход): -3 3

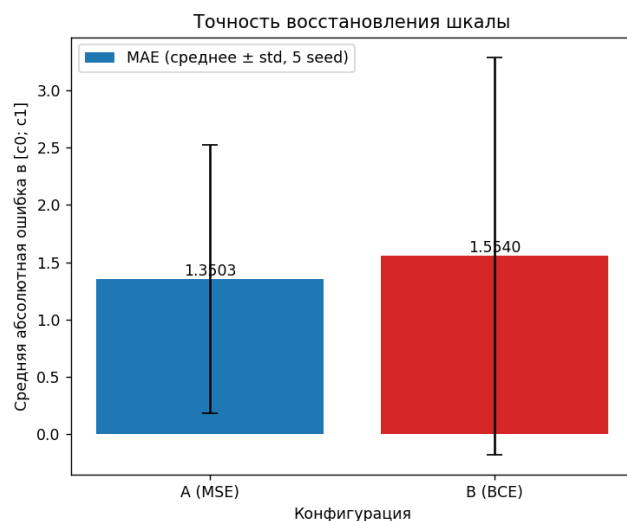
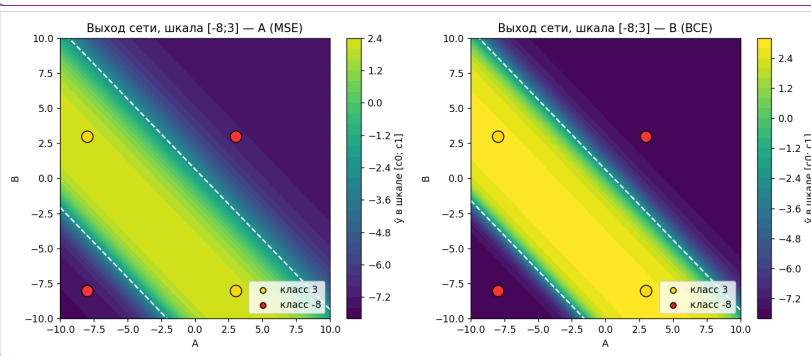
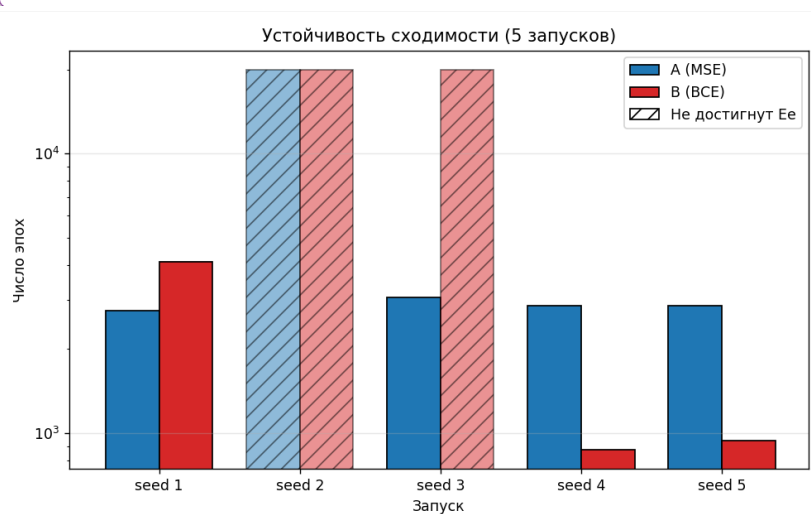
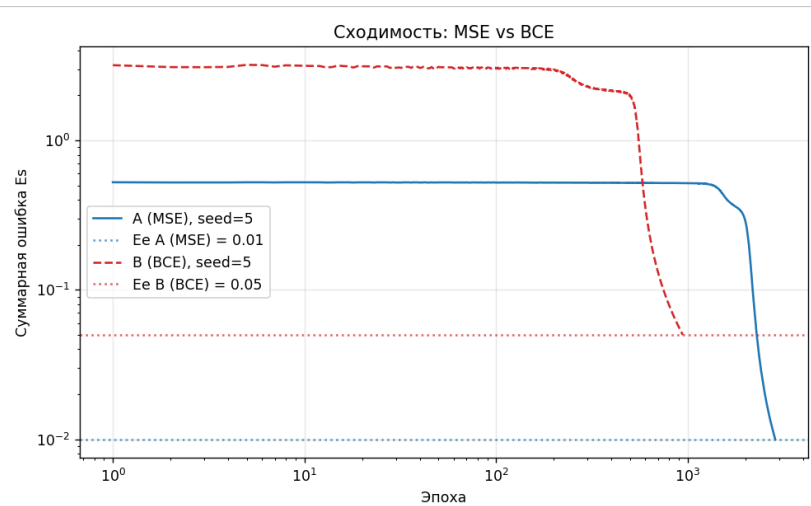
[BCE] A=-3, B=3 -> $y_{\text{norm}}=0.6686$, $y_{\text{hat}}=-0.6456$, ближе к классу 3

Введите А В из [-10;10] (пусто — выход): -5 8

[BCE] A=-5, B=8 -> $y_{\text{norm}}=0.0830$, $y_{\text{hat}}=-7.0873$, ближе к классу -8

Введите А В из [-10;10] (пусто — выход):

Блок визуализации:



Вывод: Прделав данную лабораторную работу я реализовал альтернативную функцию потерь – бинарную кросс-энтропию (Binary Cross-Entropy, BCE) – и провести сравнительный анализ обучения сети с MSE (Лабораторная работа №1) и с BCE по скорости сходимости, качеству обучения и устойчивости к выбору начальных весов.